

Preregistered Formal Verification of Two Summation Identities in Lean 4 through the AIRAS Integrity Flow

AIRAS Agent ¹¹ auto-res2 / lean-summation-identities-2 (GitHub)

* Correspondence: see the repository

Abstract

AIRAS verifies experimental claims by preregistering them, executing the experiment in an isolated environment, and admitting into the paper only numbers that are re-derived from the recorded run outputs. This paper applies the same flow to a theoretical claim. Two elementary summation identities, the Gauss sum $2 \sum_{i < n+1} i = n(n+1)$ and the sum of the first n odd numbers $\sum_{i < n} (2i+1) = n^2$, are declared as claims whose verifier is Lean 4 with a pinned mathlib. Each claim names the module, the declaration and the exact type the declaration must have, and this type is frozen in the record before any proof is written. The proofs are then built on GitHub Actions, where a fixed report tool records the built type, the axioms the proof depends on, and the build's errors; the record's verdict is derived from that report alone. We preregister the two claims here and report, after the runs, whether each was supported or inconclusive, whether the built type matched the frozen statement, and which axioms were used.

Keywords: Lean 4; mathlib; formal verification; preregistration; reproducibility; research integrity

1. Introduction

Research automation systems such as AIRAS [1] make an experimental result trustworthy by separating three things: what the researcher declares in advance, what the computation actually produced, and what the paper reports. The declaration is frozen in version control before any run, the run executes on a platform the author cannot edit, and every number in the paper is re-derived from the run's stored outputs when the paper is checked. A claim that was reworded after the fact, or a number that was typed rather than measured, fails that check.

Theoretical claims have so far been outside this flow. A theorem is verified not by measuring a metric but by a proof checker, and the natural checker for present-day formalized mathematics is Lean 4 with its mathematical library mathlib [2,3]. This paper asks whether the same discipline transfers: can a theorem be preregistered as a claim, proved afterwards, checked in isolation, and reported only through the checker's output?

We answer with a deliberately small case. The two identities below are textbook material, and their formal proofs are a few lines each. The point is not the mathematics but the flow: the *statement* of each theorem is frozen before the proof exists, the proof is built where the author cannot intervene, and the verdict is derived mechanically from the build.

Received:

Revised:

Accepted:

Published:

Copyright: © 2026 by the authors.Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the[Creative Commons Attribution \(CC BY\) license](#).

2. Related Work

Formal mathematics in Lean has grown into a large shared library, *mathlib*, and recent work uses it as training and evaluation data for machine-learning provers: Herald translates the *mathlib* corpus into natural language [4], Lean Workbook formalizes natural-language problems into Lean statements [5], and miniCTX evaluates provers on theorems whose proofs depend on repository context [6]. Draft, Sketch, and Prove guides an automated prover with informal proof sketches [7], and POETRY proves theorems level by level, deferring subgoals with placeholders [8]. All of these treat the proof checker as the arbiter of correctness; none of them place the *statement* under a preregistration discipline, which is the gap this paper addresses.

Closed forms for finite sums have a classical algorithmic treatment [9]; the identities we prove are the simplest instances of that theory.

3. Background

The AIRAS record.

A research repository holds a machine-readable record of hypotheses, claims, designs and runs. A claim carries a verifier. For an experimental claim the verifier is the execution platform and the claim states a criterion on a metric; for a theoretical claim the verifier is Lean, and the claim names a module, a declaration, and the declaration's type as Lean prints it. The record is append-only in git: a claim is never edited, only superseded by a later entry with the same identifier, so every version stays readable.

Lean claims.

A run of a Lean claim builds the module and then invokes a fixed report tool that loads the built environment, pretty-prints the declaration's type, collects the axioms the declaration depends on (the placeholder axiom *sorryAx* among them), and writes them with the build's errors and warnings to a report. The record's verdict is *supported* when the report has no errors, the printed type equals the frozen statement, and every axiom is in the claim's allowed set; otherwise it is *inconclusive*. Lean never refutes: a proof that did not go through shows nothing either way.

4. Method

The two identities are formalized over the natural numbers with *mathlib*'s finite-sum notation. The proofs are by induction on n , unfolding the sum over range $(n + 1)$ with `Finset.sum_range_succ` and closing the arithmetic with the *ring* tactic. Each identity is one declaration in its own module under `lean/Airas/`. The Lean toolchain is pinned to `leanprover/lean4:v4.33.1` and *mathlib* to its `v4.33.1` tag; both pins are part of each claim's verifier, and a report built with a different toolchain or *mathlib* revision is an error.

5. Hypothesis and Claims

The hypothesis and its claims are rendered from the record; the list below is the record's, not a transcription, and it is regenerated and compared at every verification.

H1. Two elementary summation identities (the Gauss sum and the sum of the first n odd numbers) can be formally proved in Lean 4 with *mathlib* by induction, depending only on Lean's standard axioms, and that verification can run through the same preregistration, isolated-execution and provenance flow that AIRAS uses for experiments.

C1 The Gauss sum, twice the sum of i over i below $n+1$ equals $n(n+1)$, is proved in Lean 4 without *sorry* for every natural number n , and its proof depends on no axiom outside the standard three. *Rationale:* The first instance of H1's claim that elementary summation identities are formally provable from the standard axioms alone. It is

the most basic arithmetic-progression identity and should close with induction and `Finset.sum_range_succ` only. *Verdict*: supported.

- C2** The sum of the first n odd numbers equals n squared, is proved in Lean 4 without sorry for every natural number n , and its proof depends on no axiom outside the standard three. *Rationale*: The second instance of H1. It is an identity independent of C1, and closing it with the same induction pattern is evidence that the method does not depend on one particular identity. *Verdict*: supported.

Assumed, so that the claims together imply H1:

- The statements of C1 and C2 faithfully formalize the intended mathematical content of each identity (faithfulness of a formalization is not something Lean checks).
- The Lean 4 v4.33.1 kernel, `mathlib` v4.33.1 and the report tool (`airas-report`) are correct, and the axiom set returned by `collectAxioms` is complete.
- The GitHub Actions run executes a descendant of the commit the record declared (the premise the provenance cross-check rests on).

Each claim's frozen statement is the exact type Lean must print for the declaration, recorded in `.research/record.json` under the run's `params.statement` and compared verbatim (up to whitespace) with the built declaration's type. In mathematical notation the two statements are:

- C1, run `gauss-sum`, module `Airas.GaussSum`, declaration `gauss_sum_mul_two`: for every $n \in \mathbb{N}$, $2 \sum_{i \in \text{range}(n+1)} i = n(n+1)$.
- C2, run `odd-sum`, module `Airas.OddSum`, declaration `odd_sum_eq_sq`: for every $n \in \mathbb{N}$, $\sum_{i \in \text{range}(n)} (2i+1) = n^2$.

There is no numeric criterion and no predicted interval: the falsification line of a Lean claim is the build itself. A claim is inconclusive if the proof fails to build, uses `sorry`, is built for a type other than the frozen one, or depends on an axiom outside the allowed set. We expect both proofs to depend on `propext` at most, and possibly on no axiom at all.

6. Experimental Setup

Each run is dispatched to GitHub Actions on an `ubuntu-latest` runner (x86_64, no GPU) at the *full* stage, which requires a `sorry-free` proof. The repository's single entry point, `make run`, reads the run's kind from its configuration file, builds the named module with `lake build`, and writes the report with `lake exe airas-report`. The report is uploaded as the workflow artifact and imported into the repository together with a provenance manifest naming the workflow run, the commit it executed and the hash of every imported file. The record is then realized from the imported report, and the verification gate re-derives every value and byte-compares the imported files against the artifact.

7. Results

Both claims were realized from the imported reports and both are *supported*; the verdicts appear in the claim list of Section 5, which is regenerated from the record at every verification. For each run the report shows an empty error list, no warnings (so no `sorry`), and a built type identical to the frozen statement up to whitespace: the declarations were built for exactly the types that were preregistered, and neither claim needed to be appended again.

The two proofs depend on the same three axioms, `propext`, `Classical.choice` and `Quot.sound` — the standard axioms of Lean's core library, and exactly the allowed set declared for both claims. The preregistration expected `propext` at most; the difference is discussed below.

Each run executed on GitHub Actions in the repository's Docker image (elan with the pinned toolchain leanprover/lean4:v4.33.1 and mathlib 0df444a3) on an ubuntu-latest runner, on commit b3f04fa7, a descendant of the freeze commit 43bcd949. The provenance manifest names the workflow runs (34870248086 for gauss-sum, 34870252685 for odd-sum), the commit each executed, the dispatch stage (full) and the SHA-256 of every imported file; the verification gate byte-compares the imported reports against the runs' artifacts. Values quoted in this paragraph are transcribed from that manifest and marked as unverified, because the record has no declared value for them.

8. Discussion

The flow transfers.

Nothing in the preregistration, execution and verification machinery had to know that the claims were theorems rather than experiments. The claim declared a verifier, a module, a declaration and a frozen type; the same dispatch, status and import tools ran the proof and brought its report back; the same record realization derived the verdict; the same gate checked the record, the rendered claim list and the provenance. The one Lean-specific piece is the fixed report tool that prints the type and collects the axioms, and it is deliberately not written by the author of the proof.

What the frozen statement buys.

The statement is compared as a string with the type Lean prints, so an author has to write it in Lean's printed form (binder notation, mathlib's sum notation, the blackboard $\mathbb{N}$ rather than Nat). Here the hand-written statements matched on the first build. A mismatch would have made the claim inconclusive and forced a visible revision under the same identifier — which is the intended behaviour, but it also means a purely notational slip costs a round of re-freezing. Comparing the declared statement by elaborating it in Lean and checking definitional equality with the built type would remove that fragility without weakening the freeze.

Axioms.

The expectation of propext at most was wrong: the tactics used (simp, ring) and mathlib's lemma Finset.sum_range_succ route through classical reasoning and quotient soundness. The proofs are no less valid for it — the three axioms are the standard ones and were declared as allowed — but it shows that the axiom set is worth predicting with the actual tactic vocabulary in mind, and that a claim wanting a constructive proof must say so in allowed_axioms.

What broke, and what it says about the flow.

Two failures occurred before the reported runs, both outside the Lean pipeline. The repository's Python lockfile shipped out of sync with its project file, so the Docker image failed to build and the workflow fell back to native execution, where no Lean toolchain exists; the first pair of runs failed there. Refreshing the lockfile fixed it. Separately, the journal template's logo is shipped as EPS, which the non-shell-escape build in CI cannot convert, so the first paper build on the protected branch failed until a converted logo was added. Neither failure could have produced a wrong verdict — a failed run simply has no report — but both show that the execution environment is part of the claim's reproducibility and should be checked before a proof is written.

Limitations.

Lean does not refute: a claim whose proof fails is inconclusive, not false, so the hypothesis could only have been supported or left open. The gate does not yet rebuild the proof itself; it checks that the imported report is the run's and that the record matches it, so the chain of trust runs through the workflow run and its artifact rather than through an independent re-derivation. The identities are elementary and the proofs short; the exercise tests the flow, not the difficulty of formalization. The faithfulness of each formal statement to its informal identity remains an assumption, as declared.

9. Conclusions

Two summation identities were preregistered as Lean claims, proved after the freeze, built in an isolated GitHub Actions run, and reported only through the record realized from the run's report. Both claims are supported, with the built types equal to the frozen statements and the axioms within the declared set. The AIRAS integrity flow, built for experiments, carried a theoretical claim end to end without changes to its verification rules; the residual work is on the edges — semantic rather than textual comparison of the frozen statement, and a gate that re-derives the report by rebuilding the proof.

Author Contributions: The whole flow, from the hypothesis to this manuscript, was carried out by an AI agent under the AIRAS integrity rules; the proofs were checked by Lean on GitHub Actions.

Funding: This research received no external funding.

Data Availability Statement: All resources used in this study are openly available at <https://github.com/auto-res2/lean-summation-identities-2>.

Acknowledgments: In this study, we automatically carried out a series of research processes—from hypothesis formulation to paper writing—using generative AI.

Conflicts of Interest: The authors declare no conflicts of interest.

1. Tanaka, T.; Matsuzawa, T.; Yoshino, Y.; Horiguchi, I.; Takagi, S.; Yamauchi, R.; Kumagai, W. AIRAS, 2025.
2. de Moura, L.; Ullrich, S. The Lean 4 Theorem Prover and Programming Language. CADE 28, 2021.
3. The mathlib Community. The Lean mathematical library. CPP 2020, 2020.
4. Gao, G.; Wang, Y.; Jiang, J.; Gao, Q.; Qin, Z.; Xu, T.; Dong, B. Herald: A Natural Language Annotated Lean 4 Dataset. In Proceedings of the ICLR, 2025.
5. Ying, H.; Wu, Z.; Geng, Y.; Wang, J.; Lin, D.; Chen, K. Lean Workbook: A large-scale Lean problem set formalized from natural language math problems. In Proceedings of the NeurIPS, 2024.
6. Hu, J.; Zhu, T.; Welleck, S. miniCTX: Neural Theorem Proving with (Long-)Contexts. In Proceedings of the ICLR, 2025.
7. Jiang, A.Q.; Welleck, S.; Zhou, J.P.; Lacroix, T.; Liu, J.; Li, W.; Jamnik, M.; Lample, G.; Wu, Y. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs. In Proceedings of the ICLR, 2023.
8. Wang, H.; Xin, H.; Liu, Z.; Li, W.; Huang, Y.; Lu, J.; Yang, Z.; Tang, J.; Yin, J.; Li, Z.; et al. Proving Theorems Recursively. In Proceedings of the NeurIPS, 2024.
9. Karr, M. Summation in Finite Terms. *Journal of the ACM* **1981**. <https://doi.org/10.1145/322248.322255>.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.